Introductory Programming and Problem-Solving Skills

Week - 11

Let’s Recap

- Exceptions and Exception Handling
- Raising manual Exception
- Catching single and Multiple Exception
- Try , Except, Else and Finally

Algorithms

1. Introduction

Navigation

1. Introduction

- **Almost all computation involves the execution of a series of steps. Sometimes these steps are repetitive and sometimes they are complex, requiring decision-making and branches of alternative steps.**
- **These steps are generally described as algorithms**
- **We will look at a number of standard, well known algorithms and their implementation in this Lecture.**

1.1 Searching - linear search

Navigation

1. Algorithm

1.1 Searching – Linear Search

- **If we have a string or a list of unordered, searching to see if a specific letter exist in the string is straightforward and consists of looking at every element of the list until the letter is found or we reach the end of the list :**

```
def find(c, abc):
    for i in range(len(abc)):
        if c == string[i]:
            return i
    return -1
>>> string = 'qwertyuiop'
>>> c = 'y'
>>> find(c,string)
5
>>> c = 'a'
>>> find(c,abc)
-1
```

If the char is found it returns the index of char

Else the function returns -1

Looks for chr ‘y’ returns its index i.e. 5

Returns -1, a is not present in the string

1.2 Searching - ordered string

Navigation

1. **Algorithm**

1.1 Searching – Linear Search

1.2 Searching – ordered string

If the string or list is ordered (e.g., 'abcedfgh'), there is another alternative to searching every element to look for a specific letter.

We can use the **divide-and-conquer** strategy:

1. Find the middle of the list: 'abc**d**efgh'

2. Compare the letter **'c'** to the letter in the list in the middle position: abc**d**efgh', if matched, end the search.

3. Otherwise, if the letter is lower than the letter in the list, split the list into two at that position : **'abcd'** and **'efgh'** and discard the upper half. Similarly, if the letter is higher, discard the lower half

4. Repeat from step 1 with the shortened list: 'abcd', -> 'ab' and 'cd' , -> 'cd' -> 'c' and 'd', -> 'c'

This is also known as the Binary Search

1.3 Searching - binary search

Navigation

1. Algorithm

1.1 Searching – Linear Search

1.2 Searching – ordered string

1.3 Searching – binary Search


```
def ordered_find(c, string):
    start = 0
    end = len(string)-1
    while True:
        mid = (start + end)//2
        if c == string[mid] :
            return mid
        if start == end :
            return -1
        if c < string[mid] :
            end = mid
        else :
            start = mid +1
```

- search for a character c in a sorted string.
- start index is set to 0 and the end index is set to the last index of the string (len(string) - 1)
- While loop is used until the char is found
- middle index is calculated as the average
- If the character is less than the character at the middle index, the search range is narrowed
- The loop then continues with the narrowed search range until the character is found

1.4 Linear search vs binary search

Navigation

1. Algorithm

1.1 Searching – Linear Search

1.2 Searching – ordered string

1.3 Searching – binary Search

1.4 Linear Search -VS – Binary Search

Difference

Sequential or Linear Search	Binary Search
1) Record is search sequentially.	1) Records are partitioned in two parts and it compared with middle key value.
2) Search terminates when record is found or file ends	2) Search terminates when record is found or no more partitions are possible
3) Simple method for searching.	3) Fast & Efficient method for searching.
4) We can applied on sorted or unsorted data	4) We can apply on only sorted data
5) Time Complexity: Best Case : O(1) Worst Case : O(n+1)	

Sorting

2. Sorting

Navigation

2. Sorting

Since having a ordered/sorted list can be very useful in a lot of situations in processing data, efficient algorithms for sorting data are important. When presented with an unordered list :

```
abc = ['b', 'd', 'e', 'a', 'c']
```

one of the simplest ways of sorting this list is just to go through the list from start to end, and look for the lowest letter in the list. When you reach the end of the list, swap the lowest letter with the letter at the front of the list.

```
abc = ['a', 'd', 'e', 'b', 'c']
```

Then start that process again from the second position in the list and search through the list again for the next lowest letter, and then swap that with the second position, and so on.

```
abc = ['a', 'b', 'e', 'd', 'c']
abc = ['a', 'b', 'c', 'd', 'e']
abc = ['a', 'b', 'c', 'd', 'e']
abc = ['a', 'b', 'c', 'd', 'e']
```

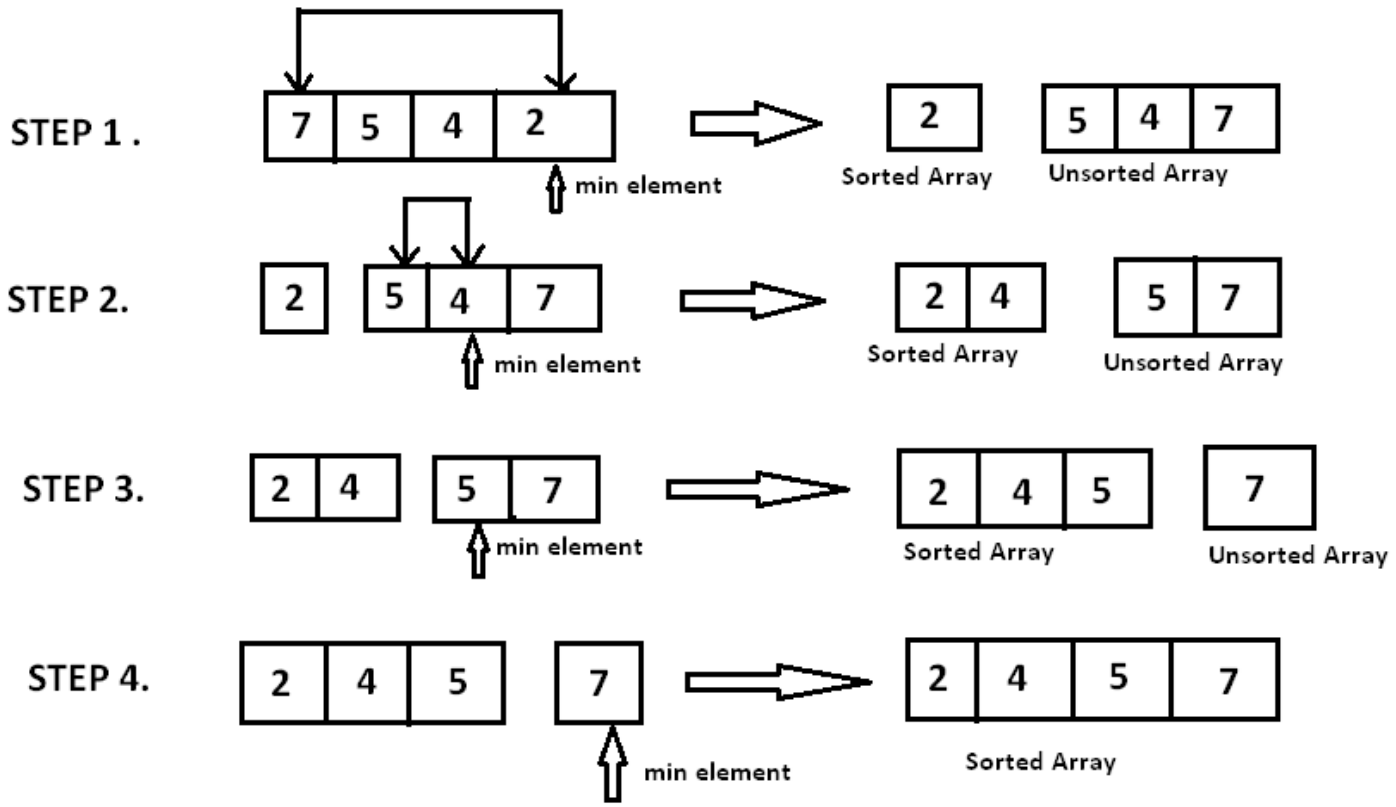
This is called the Selection Sort algorithm, and it is very inefficient. Its time complexity is $O(n^2)$.

2.1 Selection Sort

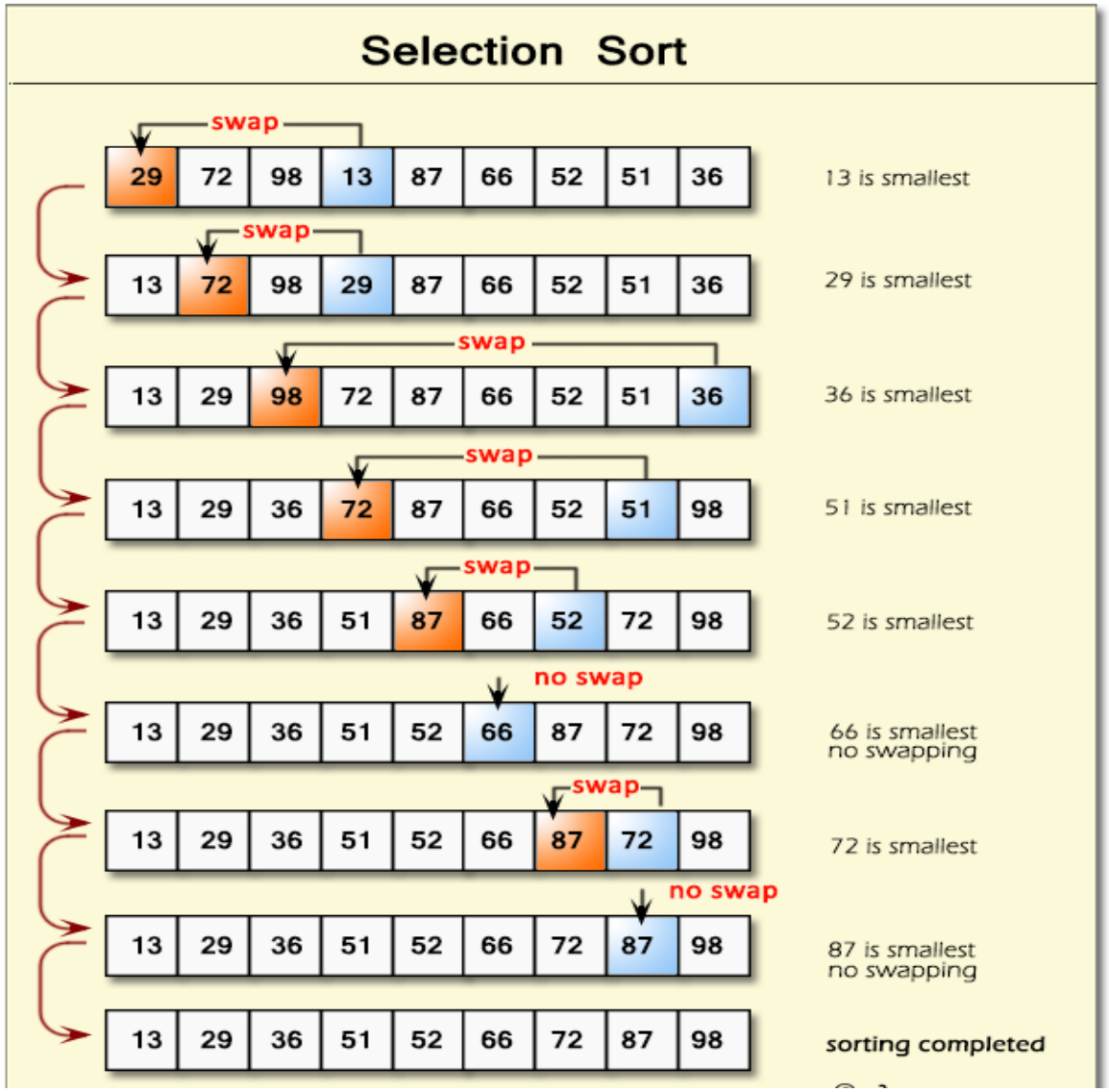
Navigation

2. Sorting

2.1 Selection Sort



2.1 Selection Sort(cont....)



Recursion

3. Recursion

Navigation

3. Recursion

Recursion is all about **"Divide and Conquer"**

Breaking a problem down to its smallest step, applying a simple solution, and then stepping back and applying the same solution to bigger and bigger versions of the problem

However, not all problems can be solved this way

Sometimes it is more elegant to solve a problem recursively, but it may not be the most efficient way.

3.1 Recursive function

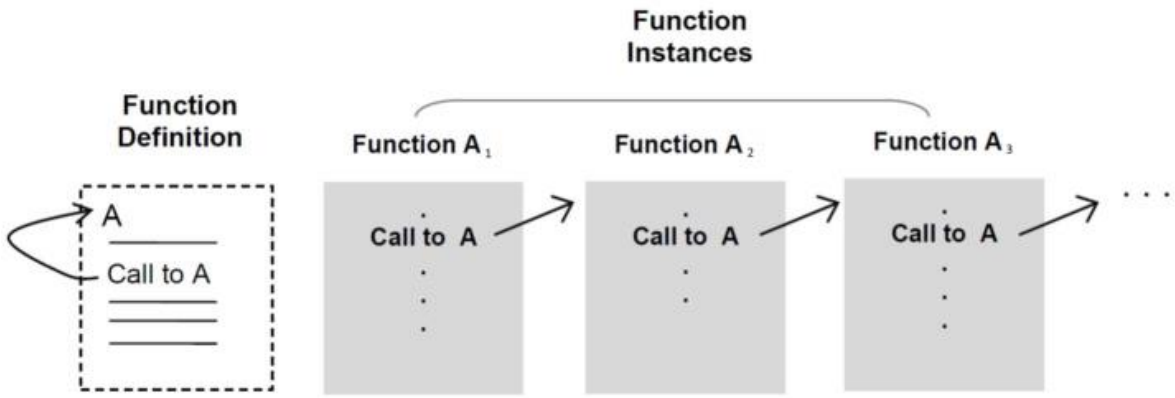
Recursion in most programming languages is achieved through functions (or methods)

In this example, Function A call itself at some point inside of the function code body, and creates a new instance of the function

Each time the code inside the function calls itself, it create another instance.

However at some point in this process, the code has to decide not to call itself again, or the code will just keep calling forever (infinite recursion - like infinite loop)

Usually this is at the point where the problem has been reduce to a single step.



3.2 Example - Function to print decreasing numbers

Navigation

3. Recursion

3.1 Recursive Functions

3.2 Function Example

```
def recursivefunc(n):
    print('n = ',n)
    if n > 0:
        recursivefunc(n - 1)
```

1. It first prints the value of n using the print() function.
2. It then checks if n is greater than 0.
3. If n is greater than 0, the function calls itself with the argument n - 1. (***This is the recursive step, where the function calls itself with a smaller value of n.)***
4. If n is not greater than 0, the function does not call itself and simply returns.

3.3 Recursive Search

Navigation

3. Recursion

3.1 Recursive Functions

3.2 Function Example

3.3 Recursive Search

```
def recur_find(c, string, start, end):
    # Calculate the middle index of the substring
    mid = int((start + end)/2)
    # If the middle character is the search character, return its index
    if c == string[mid] :
        return mid
    # If the substring has only one character and it's not the search character, return -1
    if start == end :
        return -1
    # If the search character is less than the middle character, search the left half of the substring
    if c < string[mid] :
        return recur_find(c, string, start, mid)
    # If the search character is greater than the middle character, search the right half of the substring
    else :
        return recur_find(c, string, mid+1, end)
```

3.4 Merge Sort Recursive Algorithm

Navigation

3. Recursion

3.1 Recursive Functions

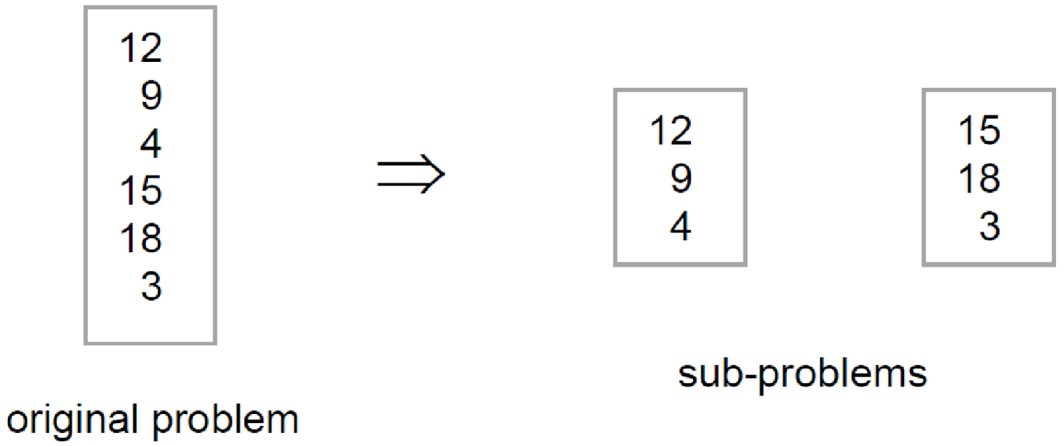
3.2 Function Example

3.3 Recursive Search

3.4 Recursive Algorithm

Earlier on we look at a simple sorting algorithm that wasn't very scalable. Now we will look at how we can apply recursive problem solving to the problem of sorting lists. In order to solve this problem recursively, we have to first imagine how the problem of sorting can be broken into subproblems, such that one or more of the subproblems is also the problem of sorting a (smaller) list.

When breaking a problem down, it is often most effective to break the problem into equal size subproblems. We consider, therefore, breaking down the problem of sorting `n` elements, into two subproblems of sorting lists of size `n/2`, as demonstrated below:



3.4 MergeSort Recursive Algorithm (cont.)

Navigation

3. Recursion

3.1 Recursive Functions

3.2 Function Example

3.3 Recursive Search

3.4 Recursive Algorithm

At this point, the power of recursive thinking comes into play. Since we are developing a method for sorting lists of size `n`, we can assume that any list of size less than `n` can be sorted, without needing to show how that is done. Thus, we can continue to develop our method for sorting lists of size `n` based on that assumption.

The next step is to identify a base case that does not need to be broken down any further in order to be solved. The obvious base case is lists of size 1, since by definition they are sorted. Since we are dividing each list into two sub-lists at each step of the recursion, eventually each sub-list will be of size 1. Thus, the breakdown into subproblems leads towards the base case.

The last step is to determine how the two sorted sub-lists can be combined into one.

3.4 MergeSort Recursive Algorithm (cont.)

Navigation

3. Recursion

3.1 Recursive Functions

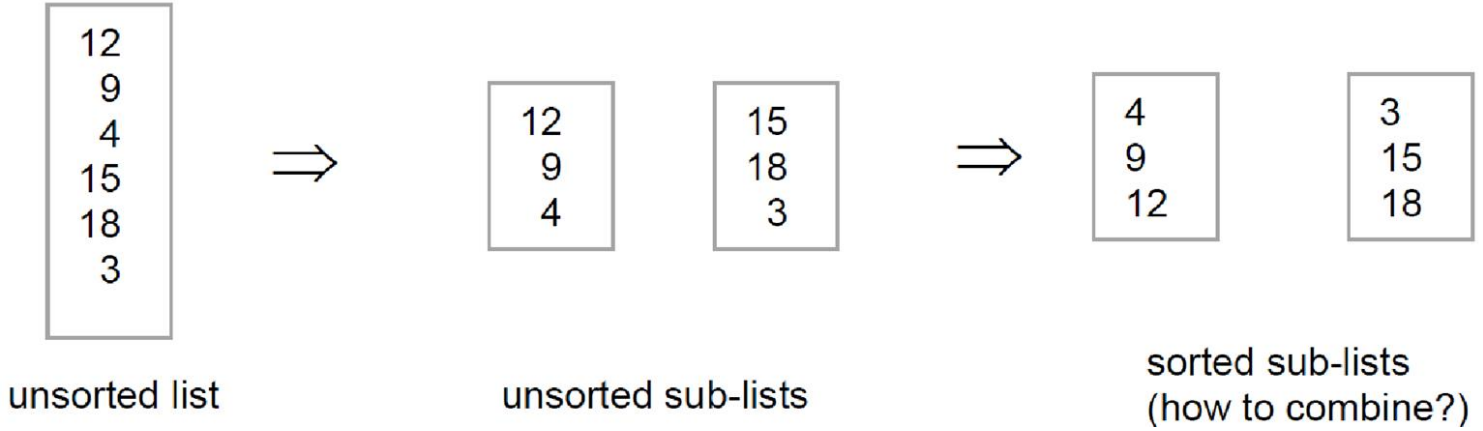
3.2 Function Example

3.3 Recursive Search

3.4 Recursive Algorithm

We cannot simply concatenate one sub-list with the other. We need to somehow merge the values into one properly sorted list.

This can be done as follows. Compare the smallest (top) item in each of the two lists. Whichever is the smaller value, move that as the first item in a new list. Cross off the item moved. Continue in the same manner until all the elements have been moved to the new list.



3.4 Merge Sort Recursive Algorithm (cont.)

Navigation

3. Recursion

3.1 Recursive Functions

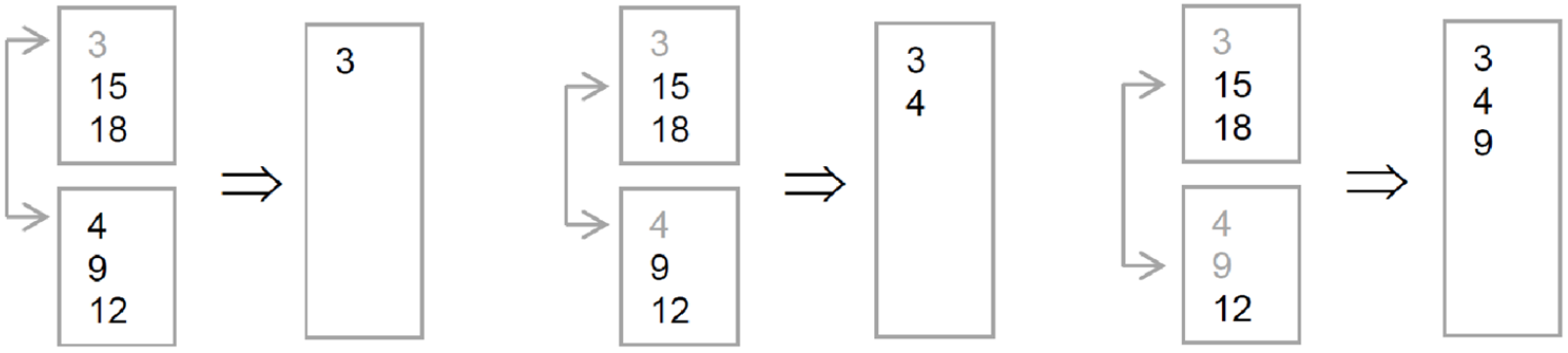
3.2 Function Example

3.3 Recursive Search

3.4 Recursive Algorithm

Now that we have determined how to break down the problem into identical (smaller) subproblems and how to combine the solutions of the subproblem into a solution of the original problem, we can specify the complete steps of this “merge sort” algorithm. This is given below.

1. Split the list into two sub-lists.
2. Sort each sub-list.
3. Merge the sorted sub-lists into one sorted list.



Merge Step 1 Merge Step 2 Merge Step 3

3.4 Merge Sort Recursive Algorithm (cont.)

Navigation

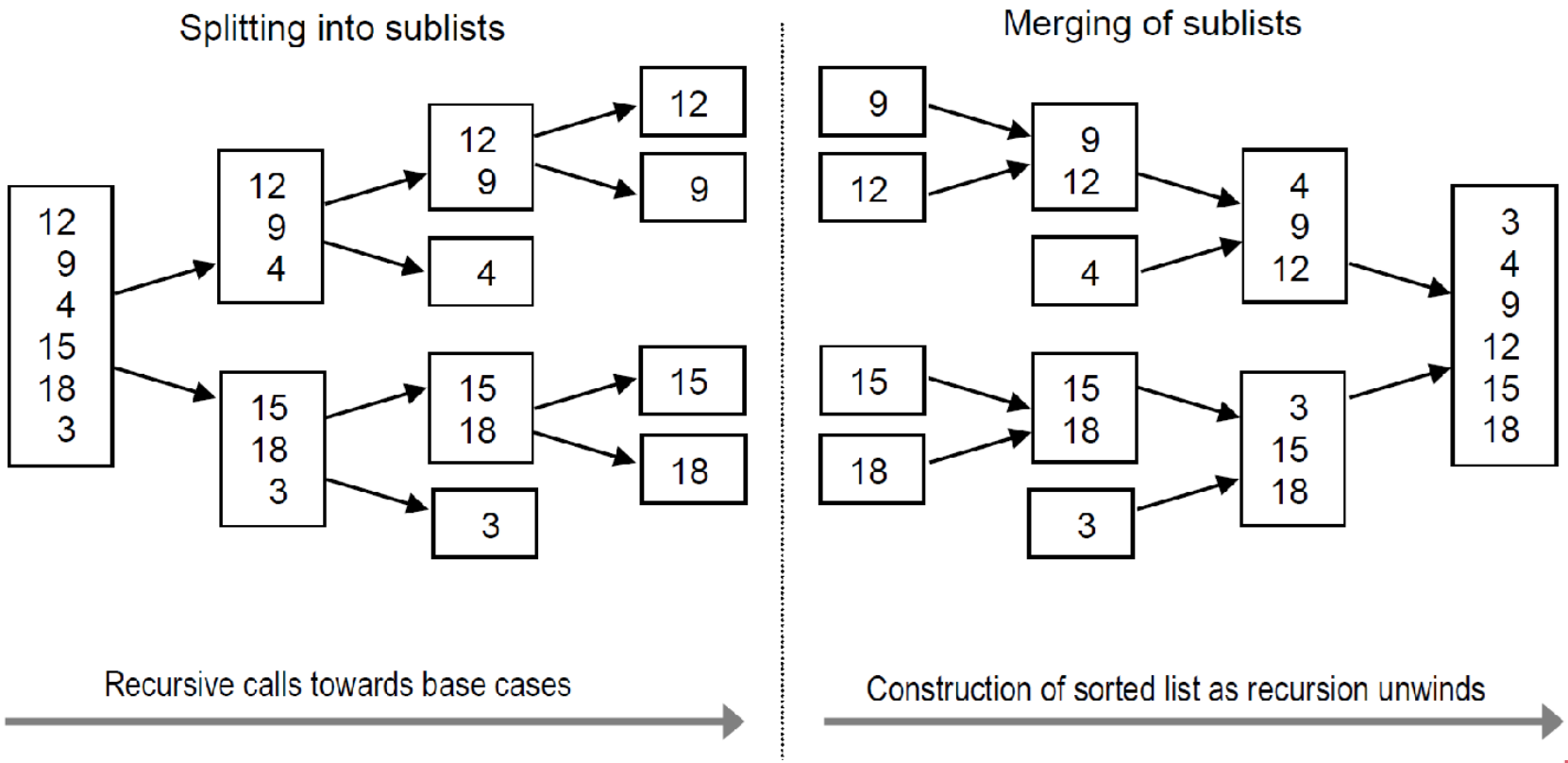
3. Recursion

3.1 Recursive Functions

3.2 Function Example

3.3 Recursive Search

3.4 Recursive Algorithm



3.4 Merge Sort Recursive Algorithm (cont.)

Navigation

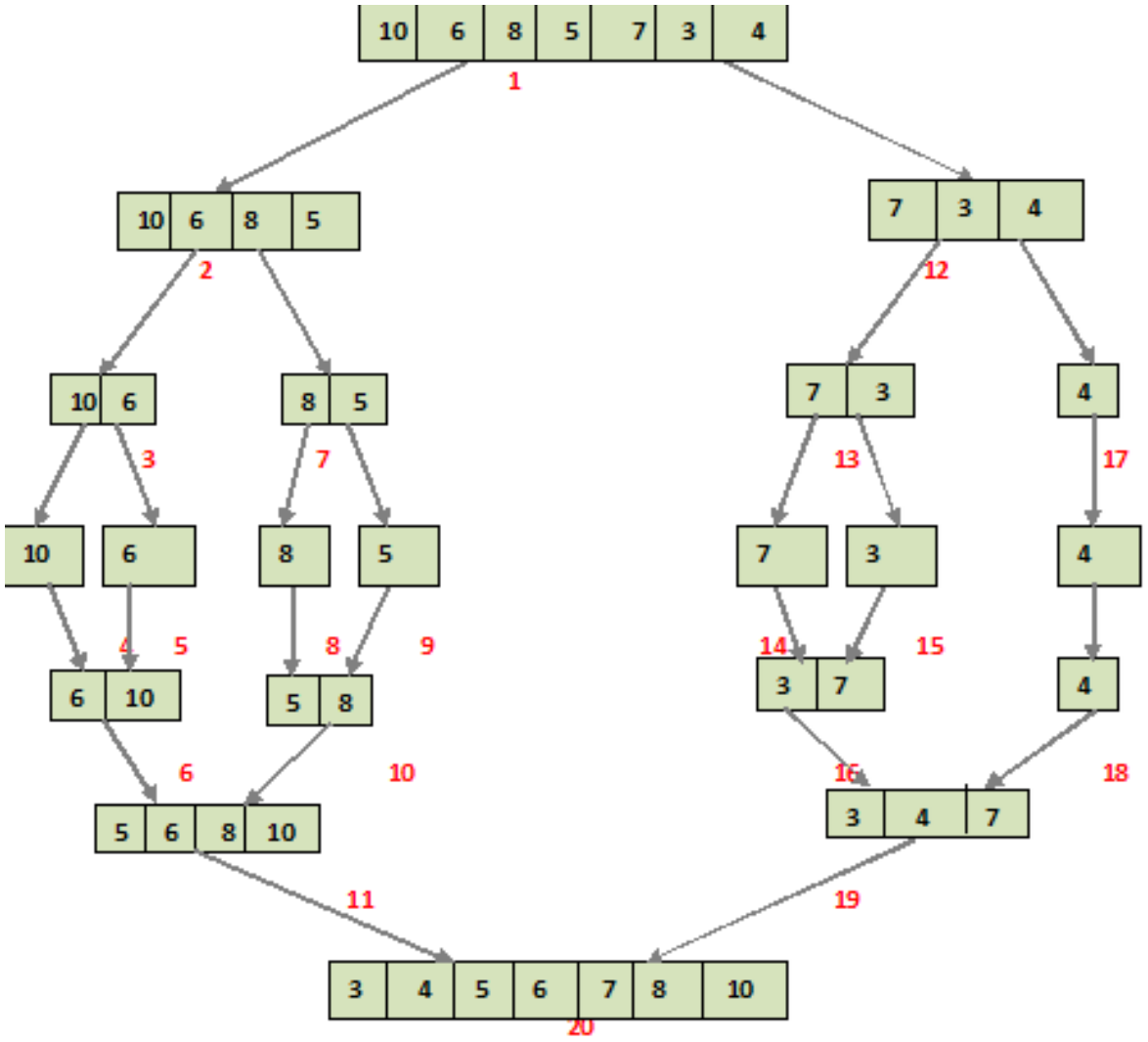
3. Recursion

3.1 Recursive Functions

3.2 Function Example

3.3 Recursive Search

3.4 Recursive Algorithm



3.4 Merge Sort Recursive Algorithm (cont.)

Navigation

3. Recursion

3.1 Recursive Functions

3.2 Function Example

3.3 Recursive Search

3.4 Recursive Algorithm


```
>>> list_to_sort = [
      3, 5, 2, 6, 9, 5, 7, 75, 43, 23, 21, 1, 23, 99, 76, 35, 47
]
>>> sorted_list = mergesort(list_to_sort)
>>> print(sorted_list)
[1, 2, 3, 5, 5, 6, 7, 9, 21, 23, 23, 35, 43, 47, 75, 76, 99]
```

3.4 Merge Sort Recursive Algorithm (cont.)

Navigation

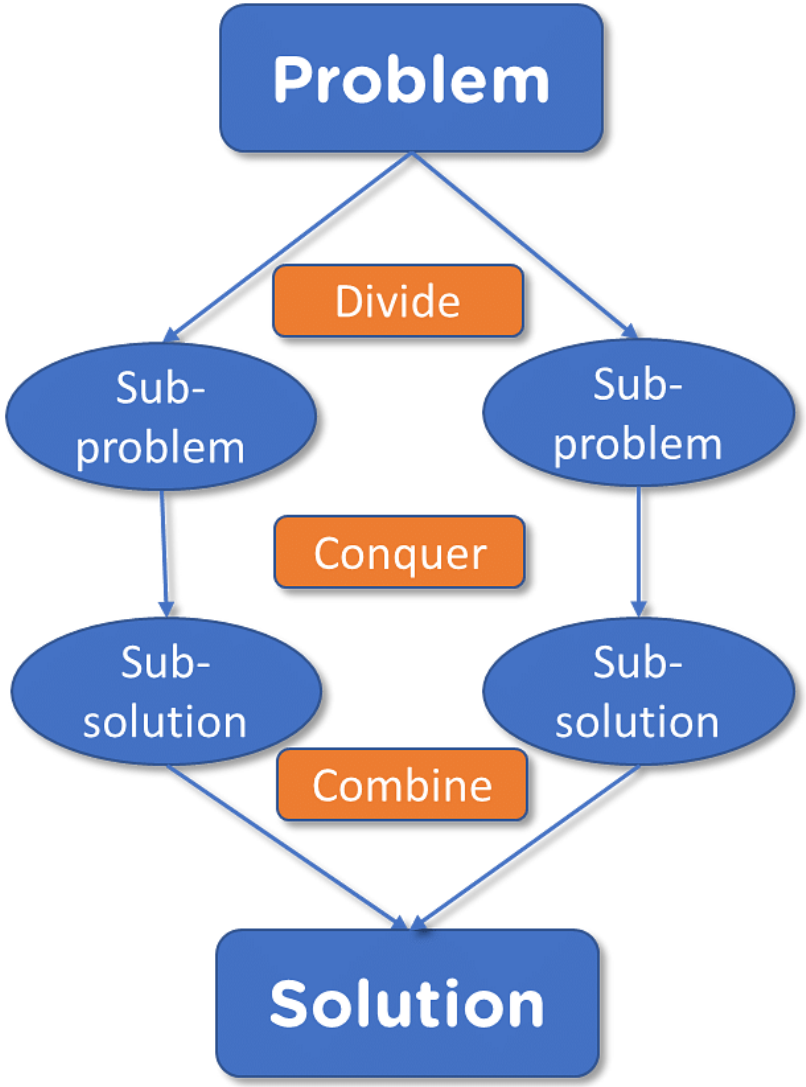
3. Recursion

3.1 Recursive Functions

3.2 Function Example

3.3 Recursive Search

3.4 Recursive Algorithm



3.5 Iteration vs. Recursion

Navigation

3. Recursion

3.1 Recursive Functions

3.2 Function Example

3.3 Recursive Search

3.4 Recursive Algorithm

3.5 Iteration vs. Recursion

RECURSION	ITERATION
Recursion function calls itself	Iteration function are repeatedly executed using conditional loop
Recursion function is terminated when the base case is reached.	Iteration function is terminated when the loop condition fails.
Recursion uses Stack memory to store the current state of the function.	Iteration function does not use stack memory.
Recursion function takes up more memory	Iteration function takes up less memory.
Infinite recursion function occurs if the recursion step does not reduce the problem in a manner that converges on the base case.	Infinite iteration function occurs if the loop condition never becomes false.
The execution of recursion is comparatively slower.	The execution of iteration is comparatively faster

UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

ing